

Section 4.2 — Relational Tuning & Database Scaling Models

Enhanced Study Notes | 5 Files in Series (4.1–4.5)

Study Directive (5 files): This is file 2 of 5 in the Section 4 database series. This section is **architecture-heavy and comparison-heavy**. Give extra weight to before/after diagrams for index strategies, replication topologies, and sharding decisions. Include specific named tools (*gh-ost*, *PgBouncer*, *Vitess*, *Citus*) and when to reach for each.

Executive Overview

A well-tuned relational database can serve millions of queries per second. The levers are: index design, read replicas, connection pooling, partitioning, and schema migrations. Each lever has a cost and a context where it applies. This section covers all five with enough depth to reason about them in production and in interviews.

1. Index Strategies — The Right Index for the Right Query

1.1 Index Type Selection

```
B+ TREE INDEX (default for all RDBMS)
  Best for: Range queries, ORDER BY, LIKE 'prefix%'
  Not for: Equality-only lookups on low-cardinality columns

CREATE INDEX idx_orders_created ON orders (created_at);
-- Supports: WHERE created_at > '2024-01-01'
-- Supports: ORDER BY created_at DESC LIMIT 20
-- Not useful: WHERE created_at = '2024-01-15' (hash index better)

HASH INDEX
  Best for: Exact equality lookups ONLY
  Not for: Ranges, sorting, LIKE
  Supported: PostgreSQL (explicit), MySQL MEMORY engine
```

```

CREATE INDEX idx_sessions_token ON sessions USING HASH (token);
-- Supports: WHERE token = 'abc123' → O(1)
-- Breaks:   WHERE token LIKE 'abc%' → hash useless

PARTIAL INDEX
Best for: Indexing only a subset of rows (skips irrelevant rows)
Massive space and speed benefit when condition is selective.

-- Only index unprocessed orders (active minority)
CREATE INDEX idx_orders_pending ON orders (created_at)
WHERE status = 'pending';

-- At 100M total orders, 10K pending:
-- Full index: 100M entries × 8 bytes = 800MB
-- Partial:   10K entries × 8 bytes = 80KB ← 10,000x smaller!

GiST INDEX (Generalized Search Tree)
Best for: Geospatial (PostGIS), full-text, JSONB paths, ranges
Not for: Simple scalar types (B-tree faster)

CREATE INDEX idx_locations_geo ON locations USING GIST (position);
-- Supports: WHERE ST_DWithin(position, ST_Point(lat,lon), 1000)

BRIN INDEX (Block Range Index)
Best for: Huge tables where data is naturally ordered (time-series, append-only)
Not for: Random-order data (BRIN useless if data not correlated with insertion order)

CREATE INDEX idx_events_ts ON events USING BRIN (created_at);
-- 1 billion row table: BRIN index = ~128KB (vs 32GB for B-tree!)
-- Works because rows are inserted in time order → time correlates with block position

```

1.2 Composite Index Column Ordering — The Critical Rule

```

Rule: Most selective + most frequently filtered column FIRST.
      For range predicates: equality columns first, range column last.

Query: SELECT * FROM orders WHERE user_id = 42 AND status = 'shipped'
      AND created_at BETWEEN '2024-01' AND '2024-03';

WRONG index: (status, created_at, user_id)
status has 4 values (low cardinality) → B-tree scans many entries before user_id

RIGHT index: (user_id, status, created_at)
user_id: high cardinality → eliminates 99.99% of rows immediately
status: further filters within user's orders
created_at: range scan on remaining rows (range must be last!)

Covering index — eliminate table heap lookups entirely:
Query: SELECT user_id, total FROM orders WHERE created_at > '2024-01-01'

```

```
CREATE INDEX idx_orders_cover ON orders (created_at) INCLUDE (user_id, total);
-- PostgreSQL INCLUDE: extra columns stored in leaf nodes only
-- Query served entirely from index – no table access ("index-only scan")
-- Cost: larger index size; Benefit: eliminates random I/O to table
```

Index selectivity rule of thumb:

- > 10% unique values: worth indexing for point lookups
- < 1% unique values: rarely worth it – full scan may be faster
- Boolean column: never index alone; only in composite (user_id, is_active)

1.3 Materialized Views — Pre-computed Query Results

Problem: Expensive aggregation run repeatedly by many users.

```
SELECT region, SUM(total) as revenue, COUNT(*) as orders
FROM orders
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY region;
-- 500M rows, runs every 30 seconds from dashboard → DB overwhelmed
```

Solution – Materialized View:

```
CREATE MATERIALIZED VIEW regional_revenue_30d AS
SELECT region,
       SUM(total) AS revenue,
       COUNT(*) AS orders,
       AVG(total) AS avg_order
FROM orders
WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY region;

CREATE UNIQUE INDEX ON regional_revenue_30d (region);

-- Refresh strategy options:
-- Option A: Full refresh every hour (simple, stale up to 60 min)
REFRESH MATERIALIZED VIEW CONCURRENTLY regional_revenue_30d;
-- CONCURRENTLY: readers not blocked during refresh

-- Option B: Trigger-based incremental update
-- More complex, near-real-time freshness
```

When to use materialized views:

- ✓ Dashboard queries (ok with 5-60 min staleness)
- ✓ Aggregations over millions of rows
- ✗ Repeated joins that don't change frequently
- ✗ Real-time requirements (use streaming instead)
- ✗ Data that changes every second

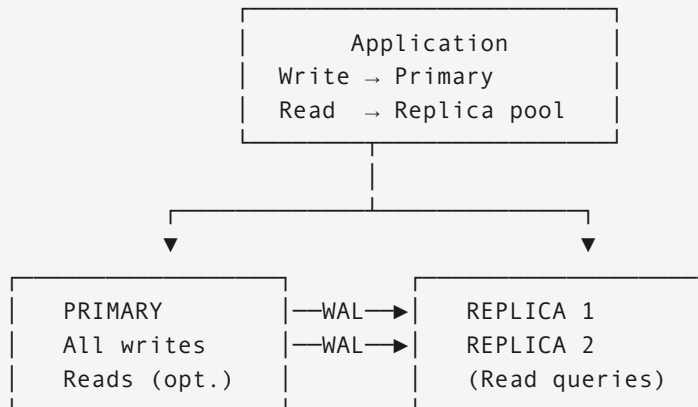
2. Replication Paradigms

2.1 Master-Slave (Primary-Replica) Replication

BEFORE (single node):

All reads + writes → single DB → bottleneck + SPOF

AFTER (primary + replicas):



Replication lag:

Async replication: 0ms write latency, 50-500ms lag possible

Semi-sync: waits for 1 replica ACK, +10ms write latency, no lag on that replica

Replication lag problem:

User writes profile update → primary

User immediately reads profile → routed to replica (lagging) → sees old data!

Mitigation strategies:

1. "Read your own writes": always read from primary for 1s after a write
2. Session stickiness: pin user's reads to same replica
3. Sync replication: guarantee zero lag (higher write latency)

PostgreSQL streaming replication modes:

synchronous_commit = on → WAL flushed to replica before ACK (safe, slow)

synchronous_commit = remote_write → WAL sent but not flushed (~safe, faster)

synchronous_commit = off → no wait (fast, committed data can be lost on crash)

2.2 Multi-Master (Multi-Leader) Replication

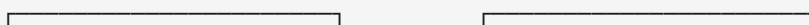
USE CASE: Multiple geographic regions each need low-latency writes.

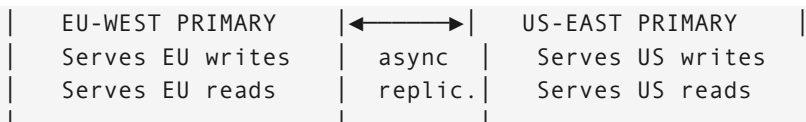
Europe users write to EU-WEST node.

US users write to US-EAST node.

Both nodes replicate to each other asynchronously.

Architecture:





CONFLICT SCENARIO:

t=0ms: EU user updates username to "alice_eu"
t=0ms: US user (same account!) updates username to "alice_us"
Both writes succeed locally.
t=500ms: EU receives US write. US receives EU write. CONFLICT.

Resolution strategies:

Last-Write-Wins (LWW):

Keep the update with the higher timestamp.
Risk: NTP drift $\pm 10\text{ms}$ → wrong winner

Application merge:

Conflict handler decides per field.
username: LWW (only one can win)
cart_items: union (merge both sets)

Custom CRDT:

Data type designed to merge without conflict.
Counter: sum all increments from all regions
Set: union of all additions minus all removals

WHEN TO USE multi-master:

- ✓ Multi-region active-active (need writes in all regions)
- ✓ Offline-first mobile apps (sync later)
- ✗ Financial data (use single-region with replicas instead)
- ✗ Strong consistency required (use distributed SQL: CockroachDB)

Replication topology comparison:

Topology	Write Latency	Consistency	Failover	Conflict
Primary-Replica	Low	Strong	Manual	None
Multi-Master	Low (local)	Eventual	Automatic	Possible
Distributed SQL	Medium (+RTT)	Strong	Automatic	None (Raft)

3. Sharding vs Partitioning

3.1 Vertical Partitioning — Split Wide Tables

PROBLEM — Wide table with mixed access patterns:

```
CREATE TABLE users (
  id          BIGINT PRIMARY KEY,
  name        TEXT,           ← read on every API call (hot)
  email       TEXT,           ← read on every API call (hot)
  bio         TEXT,           ← read only on profile page (warm)
```

```

    avatar_url    TEXT,           ← read only on profile page (warm)
    settings      JSONB,          ← 5KB average, rarely read (cold)
    raw_profile    BYTEA          ← 50KB binary blob (very cold)
);

```

Problem: Each row = ~60KB. Buffer pool polluted with cold data.
 Query for name+email loads entire 60KB row into memory.

SOLUTION – Split by access temperature:

```

CREATE TABLE users_core (      ← hot: every request
    id        BIGINT PRIMARY KEY,
    name      TEXT,
    email     TEXT
); -- 50 bytes/row

```

```

CREATE TABLE users_profile (   ← warm: profile page only
    user_id   BIGINT PRIMARY KEY REFERENCES users_core(id),
    bio       TEXT,
    avatar_url TEXT
); -- 200 bytes/row

```

```

CREATE TABLE users_extended (  ← cold: rarely accessed
    user_id   BIGINT PRIMARY KEY REFERENCES users_core(id),
    settings  JSONB,
    raw_profile BYTEA
); -- 55KB/row

```

Result: users_core buffer pool: 50 bytes × 1M users = 50MB (fits in RAM!)
 Previously: 60KB × 1M users = 60GB (could not cache)

3.2 Horizontal Sharding — Scale Beyond One Node

RANGE SHARDING:

```

Shard 1: user_id 1 - 1,000,000
Shard 2: user_id 1,000,001 - 2,000,000
Shard 3: user_id 2,000,001 - 3,000,000

```

Problem: Early users (low IDs) are most active → Shard 1 is hot!

Problem: Adding Shard 4 requires splitting an existing range → complex

HASH SHARDING:

```
shard_id = hash(user_id) % N
```

Guarantees uniform distribution regardless of access patterns.

Problem: Range queries span all shards → scatter-gather (slow)

Problem: Resharding when N changes → all keys remap (use consistent hash instead)

DIRECTORY-BASED SHARDING:

Lookup table: user_id → shard_id

Flexibility: Move specific users to different shards (hot user migration)

Cost: Lookup table is itself a bottleneck / SPOF unless highly available

SHARDING TOOLS COMPARISON:

Tool	DB	Transparency	Resharding	Cross-shard join
Citus	PostgreSQL	Transparent	Limited	Colocation-based
Vitess	MySQL	Mostly	Online	Limited
PlanetScale	MySQL	Full	Online	No
CockroachDB	Built-in	Full	Automatic	Yes (distributed SQL)

WHEN TO SHARD:

Single node limit reached: >500GB data OR >50K writes/sec
Read replicas exhausted: query load exceeds single primary
Compliance: data residency requires geographic sharding
Consider before sharding: partitioning (same node), caching, read replicas

3.3 PostgreSQL Table Partitioning (Before Sharding)

PostgreSQL native partitioning – same node, multiple physical tables:

```
CREATE TABLE orders (  
  id          BIGINT,  
  user_id     BIGINT,  
  total       NUMERIC,  
  created_at  TIMESTAMP  
) PARTITION BY RANGE (created_at);
```

```
CREATE TABLE orders_2024_q1 PARTITION OF orders  
  FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');
```

```
CREATE TABLE orders_2024_q2 PARTITION OF orders  
  FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
```

Benefits:

Partition pruning: WHERE created_at > '2024-03-01' scans only Q2+ partitions
Partition drop: DROP TABLE orders_2022 ← instant vs deleting 500M rows
Vacuum efficiency: vacuum each partition independently

When to partition vs shard:

Partitioning: table > 100GB, single node has capacity, time-based archival
Sharding: node is the bottleneck (CPU/RAM/IO), need horizontal scale

4. Connection Pooling

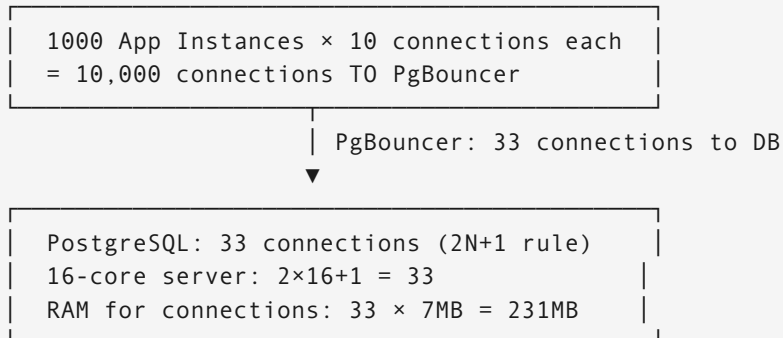
4.1 Why Connection Pooling Matters

Problem – PostgreSQL connection overhead:

Each connection = 5-10MB RAM + dedicated backend process
1000 app instances × 10 connections each = 10,000 connections

PostgreSQL: $10,000 \times 7\text{MB} = 70\text{GB}$ RAM just for connection overhead
DB performance collapses before any queries run.

Solution – PgBouncer (connection pooler):



PgBouncer pooling modes:

Session pooling: 1 DB connection held per client session (least efficient)

Transaction pooling: DB connection returned to pool after each transaction
(recommended)

Statement pooling: DB connection returned after each statement (most efficient,
limited use)

Transaction mode caveat: cannot use SET, LISTEN/NOTIFY, prepared statements
(these require session-level state)

2N+1 rule for pool sizing:

N = number of DB CPU cores

Optimal pool = $2N + 1$ (handles CPU-bound + I/O wait overlap)

16 cores → 33 connections

32 cores → 65 connections

Going above this: connections queue and wait → worse p99 latency

5. Online Schema Migrations

5.1 The Problem — Metadata Locks

Naive ALTER TABLE on a large table:

```
ALTER TABLE users ADD COLUMN last_login TIMESTAMP;
-- On a 500GB table with 200M rows:
-- MySQL: acquires EXCLUSIVE metadata lock → blocks ALL reads + writes
-- Duration: 2-6 hours for large tables
-- Downtime: effectively required
```

```
ALTER TABLE users ADD INDEX idx_email (email);
```



```
-- MySQL: table rebuild (copies entire table)
-- Still blocks writes during final cutover phase
```

5.2 Migration Tools Compared

PT-ONLINE-SCHEMA-CHANGE (Percona, MySQL):

Mechanism:

1. Create shadow table with new schema
2. Copy rows in 1000-row chunks from original to shadow
3. Install triggers on original (INSERT/UPDATE/DELETE sync to shadow)
4. When 100% copied: RENAME shadow → original (atomic, ~1s lock)
5. Drop original table, remove triggers

Pros: Battle-tested, widely used, simple

Cons: Trigger overhead on writes during migration (~10% write slowdown)
Triggers can cause replication lag on replicas

GH-OST (GitHub, MySQL):

Mechanism:

1. Create ghost table with new schema
2. Connect as replica, read binary log (not triggers)
3. Apply binlog events to ghost table + copy rows in background
4. Throttle automatically based on replica lag / Threads_running
5. Atomic cutover: brief lock (~1s) to rename tables

gh-ost \

```
--alter="ADD COLUMN display_name TEXT" \
--table=users \
--max-load="Threads_running=25" \      ← pause if DB stressed
--critical-load="Threads_running=100" \ ← abort if critical
--chunk-size=1000 \
--throttle-control-replicas=replica1
```

Pros: No triggers (reads binlog), testable/pausable, GitHub uses it in prod

Cons: Requires binlog row format, more complex setup than pt-osc

POSTGRESQL APPROACH:

PostgreSQL ALTER TABLE ADD COLUMN:

With a default: PostgreSQL 11+ → metadata-only change (instant!)

```
ALTER TABLE users ADD COLUMN last_login TIMESTAMP DEFAULT NULL;
```

← Instant even on 500M rows (PostgreSQL 11+)

With computed non-null default (old behavior): still rewrites table

Workaround: add nullable → backfill → add constraint (expand-contract)

EXPAND-CONTRACT PATTERN (works on any DB):

Phase 1 EXPAND – add, don't constrain:

```
ALTER TABLE users ADD COLUMN display_name TEXT; ← fast (nullable)
-- Deploy app code that WRITES to both old_name and display_name
-- Backfill existing rows in background (1000/batch, 100ms sleep)
```

Phase 2 CONTRACT – constrain after backfill complete:

```
-- Verify: SELECT COUNT(*) FROM users WHERE display_name IS NULL; → 0
```

```
ALTER TABLE users ALTER COLUMN display_name SET NOT NULL;
ALTER TABLE users DROP COLUMN old_name;
```

Migration tools summary:

Tool	DB	Mechanism	Write Impact	Best For
gh-ost	MySQL	Binlog replay	Minimal	Large tables, MySQL
pt-osc	MySQL	Triggers	~10% overhead	Simpler setup
pglogical	Postgres	Logical repl.	Minimal	Large PG tables
Built-in DDL	Postgres	WAL-level	None (PG11+)	Add nullable column

6. Interview Use Cases & Design Problems

Problem 1 — Slow Dashboard Query on 500M Row Table

Problem statement: An analytics dashboard runs this query every 10 seconds: `SELECT region, SUM(revenue) FROM orders WHERE created_at > NOW() - INTERVAL '30 days' GROUP BY region;` It takes 45 seconds. How do you fix it without changing the schema permanently?

Solution:

Step 1 — Diagnose with EXPLAIN ANALYZE:

```
EXPLAIN (ANALYZE, BUFFERS) SELECT region, SUM(revenue)
FROM orders WHERE created_at > NOW() - INTERVAL '30 days'
GROUP BY region;
```

Likely plan: Seq Scan on orders (cost=0..800000 rows=50M)

Problem: No index on created_at, or planner chose seq scan anyway.

Step 2 — Add partial index for recent data:

```
CREATE INDEX CONCURRENTLY idx_orders_recent
ON orders (created_at, region, revenue)
WHERE created_at > '2024-01-01'; -- adjust to fixed anchor date
-- CONCURRENTLY: doesn't lock table during build
```

New plan: Index Scan (cost=0..500 rows=2M)

Time: 45s → ~2s

Step 3 — Materialized view for the dashboard:

```
CREATE MATERIALIZED VIEW regional_revenue_30d AS
SELECT region, SUM(revenue) AS total, COUNT(*) AS orders
FROM orders WHERE created_at > NOW() - INTERVAL '30 days'
GROUP BY region;
```

```
REFRESH MATERIALIZED VIEW CONCURRENTLY regional_revenue_30d;
-- Scheduled every 5 minutes via pg_cron or Airflow
```

Dashboard query: `SELECT * FROM regional_revenue_30d;` ← instant

When to use each fix:

Index alone: works if query runs infrequently (< 1/min), data freshness required

Materialized view: correct for dashboards (slight staleness acceptable)

Both: belt-and-suspenders – index for ad-hoc queries, MV for dashboard

Problem 2 — Zero-Downtime Column Addition on 100M Row Table

Problem statement: Add `display_name TEXT NOT NULL DEFAULT (first_name || ' ' || last_name)` to a 100GB users table on MySQL. Zero downtime required.

Solution:

WRONG approach (causes downtime):

```
ALTER TABLE users ADD COLUMN display_name TEXT NOT NULL
```

```
DEFAULT (CONCAT(first_name, ' ', last_name));
```

← MySQL: rewrites entire 100GB table. 4-8 hours. Full downtime.

RIGHT approach – Expand-Contract with gh-ost:

Phase 1 (30 min – schema change with gh-ost):

```
gh-ost \
```

```
--alter="ADD COLUMN display_name TEXT" \ ← nullable first!
```

```
--table=users \
```

```
--chunk-size=1000 \
```

```
--max-load="Threads_running=30" \
```

```
--execute
```

Phase 2 (2 hours – backfill in background):

```
# Python backfill script running during business hours
```

```
cursor = 0
```

```
while True:
```

```
    rows = db.execute("""
```

```
        SELECT id, first_name, last_name FROM users
```

```
        WHERE id > %s AND display_name IS NULL
```

```
        ORDER BY id LIMIT 1000
```

```
    """, cursor)
```

```
    if not rows: break
```

```
    for row in rows:
```

```
        db.execute("UPDATE users SET display_name = %s WHERE id = %s",
```

```
                    f"{row.first_name} {row.last_name}", row.id)
```

```
    cursor = rows[-1].id
```

```
    time.sleep(0.05) # yield 50ms between batches
```

Phase 3 (seconds – add NOT NULL constraint):

```
# Verify complete:
```

```
SELECT COUNT(*) FROM users WHERE display_name IS NULL; -- must be 0
```

```
# PostgreSQL (fast with NOT VALID trick):
```

```
ALTER TABLE users ADD CONSTRAINT display_name_not_null
```

```
CHECK (display_name IS NOT NULL) NOT VALID;
```

```
-- NOT VALID: constraint not enforced on existing rows (instant)
```

```
ALTER TABLE users VALIDATE CONSTRAINT display_name_not_null;
-- VALIDATE: checks existing rows, no lock, can run online in PG
```

Phase 4 – Deploy app code using display_name.

Total downtime: 0 seconds. Total elapsed time: ~3 hours.

Problem 3 — Read Replica Lag Causing User-Visible Issues

Problem statement: Your app routes reads to replicas to reduce primary load. Users report: they update their avatar, refresh the page, see the old avatar. Replica lag is 2 seconds. Fix this without removing replicas.

Solution:

The problem – "read your own writes" violation:

```
t=0s: User uploads avatar → write to PRIMARY
t=0s: App responds "success", renders new page
t=0s: New page loads → avatar GET → routed to REPLICA (lagging 2s)
t=0s: Replica serves old avatar → user confused
```

Strategy 1 – Sticky reads after writes (simplest):

On every write, set a cookie: X-Read-Primary: expires=<now+5s>

Read routing middleware:

```
if request.cookie["X-Read-Primary"] exists and not expired:
    route to PRIMARY
else:
    route to REPLICA
```

Cost: Primary handles ~10% more reads (users who recently wrote)

Works: 100% of "read your own write" cases fixed

Strategy 2 – Session consistency via replication position:

After write, store the WAL position (LSN) in user's session:

```
session["min_lsn"] = db.execute("SELECT pg_current_wal_lsn()").scalar()
```

On each read, check replica lag:

```
replica_lsn = db.execute("SELECT pg_last_wal_replay_lsn()").scalar()
if replica_lsn >= session["min_lsn"]:
    use REPLICA ← replica has caught up, safe to read
else:
    use PRIMARY ← replica still lagging, use primary
```

Cost: 1 lightweight LSN query per request on replica

Works: Precise – only falls back to primary when actually needed

Strategy 3 – Reduce replica lag (fix root cause):

pg_stat_replication view shows lag per replica:

```
SELECT * FROM pg_stat_replication; ← shows write_lag, flush_lag, replay_lag
```

Common causes of lag:

Long-running queries on replica → lock conflicts with replay
→ Solution: `max_standby_streaming_delay = 30s` (allow replica to pause replay)

WAL sender slow → network or replica IO bottleneck
→ Solution: `wal_compression = on` (reduce bandwidth)
→ Solution: Upgrade replica to NVMe (replay is I/O bound)

When to use each:

Strategy 1: Simple, works for 80% of apps, minimal code change

Strategy 2: When primary must be protected at all costs

Strategy 3: Always investigate lag root cause first